Analyzing Index Performance Impact

Name: Zedekiah Heteroza

Student ID: 24-0521-842

INTRODUCTION

This lab activity explores how database indexes improve query performance in MySQL. The experiment compares query execution times before and after creating an index on a table column. By analyzing execution plans using the EXPLAIN statement, the activity demonstrates how indexes reduce full table scans and optimize data retrieval. The lab also examines situations where indexes are effective and where they may not significantly improve performance.

PROCEDURE

Step 1: Create Database

```
CREATE DATABASE index_performance_lab;
USE index_performance_lab;
```

A database named index_performance_lab was created and selected for use in the experiment.

Step 2: Create Table

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

A table named users was created to store user information. The id column serves as the primary key.

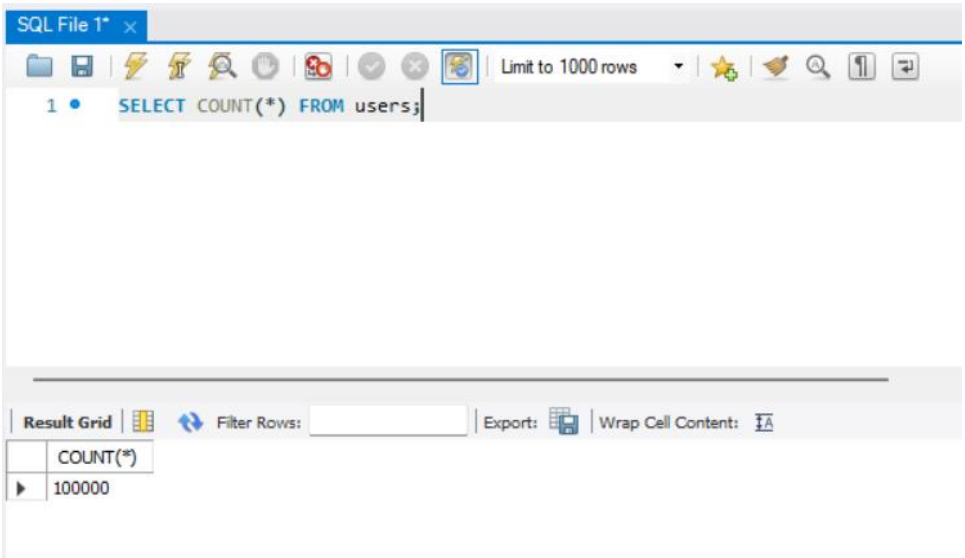
Step 3: Insert Data

```
DELIMITER //
CREATE PROCEDURE insert_users(IN num_rows INT)
BEGIN
    DECLARE i INT DEFAULT 1;

    WHILE i <= num_rows DO
        INSERT INTO users(username, email)
        VALUES (
            CONCAT('user', i),
            CONCAT('user', i, '@example.com')
        );

        SET i = i + 1;
    END WHILE;
END //
DELIMITER ;

CALL insert_users(100000);
```

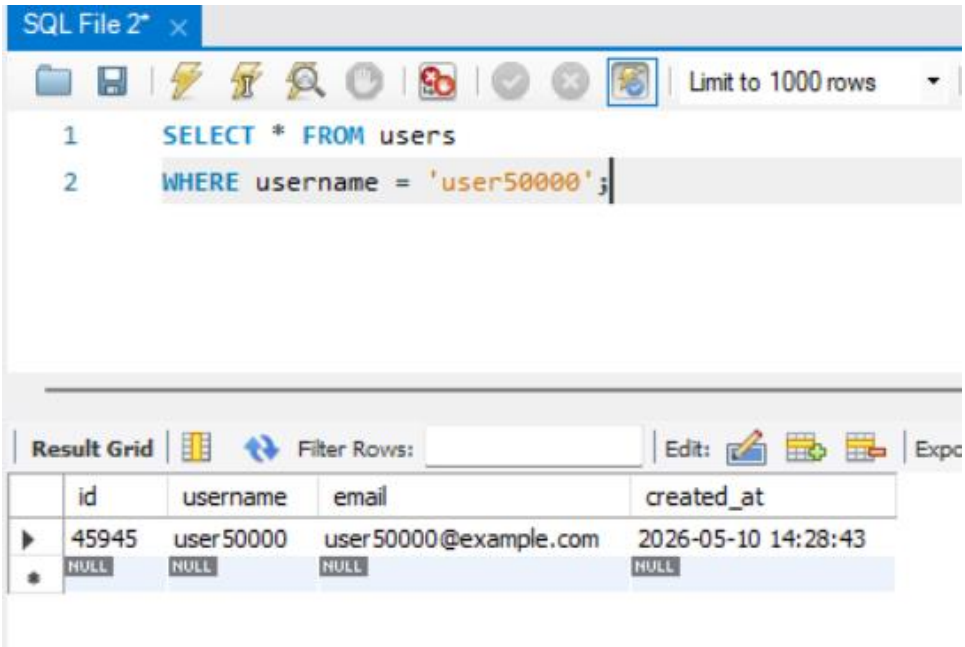


100,000 rows were inserted into the users table to simulate a large real-world dataset. A large dataset is necessary to clearly observe the performance impact of indexes.

Step 4: Query Without Index

```
SELECT * FROM users
WHERE username = 'user50000';
```

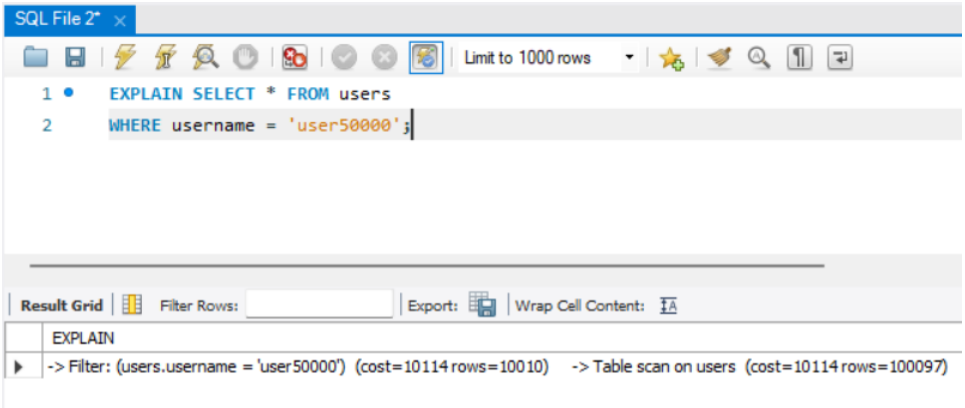
The query searched for a specific username before any index was created. Since no index existed, MySQL performed a full table scan.

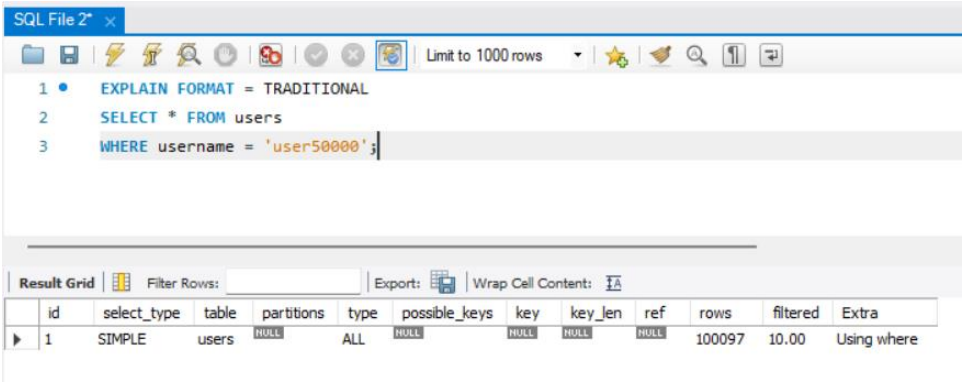


Step 5: EXPLAIN Without Index

```
EXPLAIN SELECT * FROM users
WHERE username = 'user50000';
```

```
EXPLAIN FORMAT = TRADITIONAL
SELECT * FROM users
WHERE username = 'user50000';
```





The execution plan showed a table scan on the users table, indicating that MySQL checked every row to find the matching record. This confirms that no index was used.

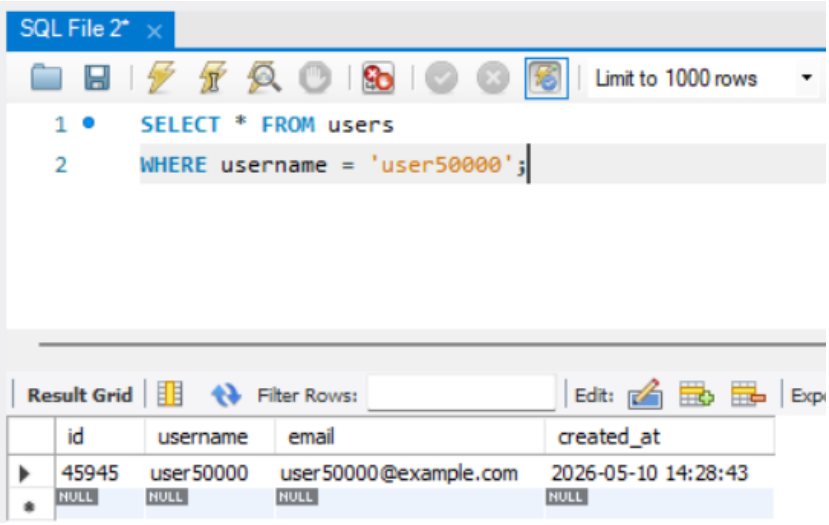
Step 6: Create Index

```
CREATE INDEX idx_username
ON users(username);
```

An index named idx_username was created on the username column to improve search performance.

Step 7: Query With Index

```
SELECT * FROM users
WHERE username = 'user50000';
```

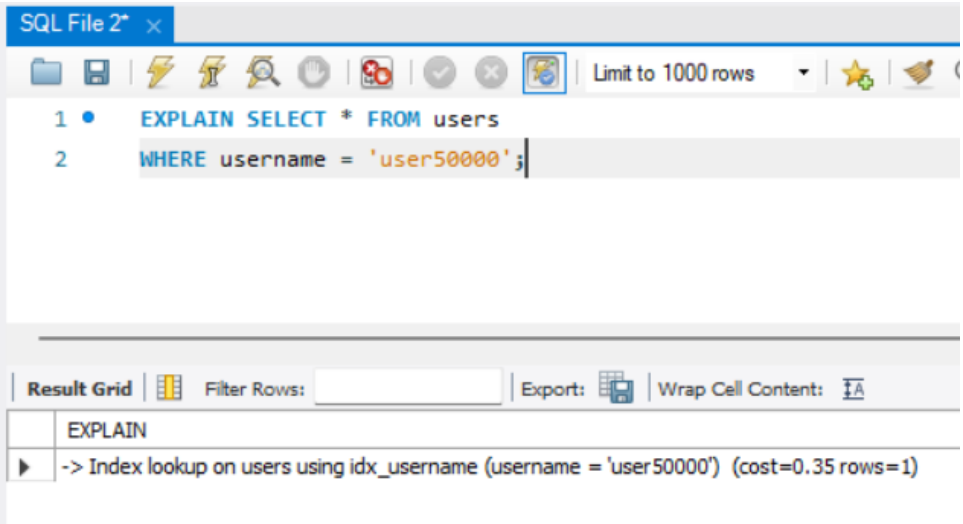


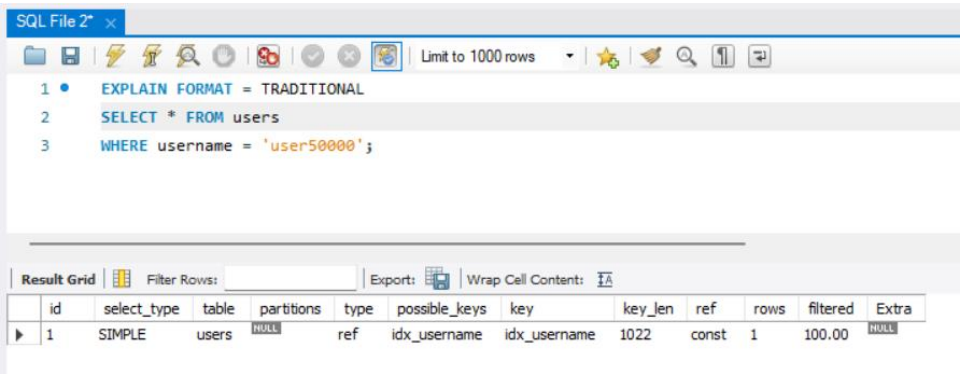
The same query was executed after creating the index. Query execution became significantly faster because MySQL used the index to directly locate the matching row.

Step 8: EXPLAIN With Index

```
EXPLAIN SELECT * FROM users
WHERE username = 'user50000';
```

```
EXPLAIN FORMAT = TRADITIONAL
SELECT * FROM users
WHERE username = 'user50000';
```





The execution plan showed that MySQL used the idx_username index instead of scanning the entire table. This improved query efficiency and reduced the number of scanned rows.

RESULTS

QUERY	EXECUTION TIME
Query without index	0.032 sec
EXPLAIN without index	0.000 sec
Query with index	0.000 sec
EXPLAIN with index	0.000 sec

6	14:29:21	SELECT * FROM users WHERE username = 'user50000' LIMIT 0, 1000	1 row(s) returned	0.032 sec / 0.000 sec
7	14:32:00	EXPLAIN SELECT * FROM users WHERE username = 'user50000'	1 row(s) returned	0.000 sec / 0.000 sec
8	14:32:22	EXPLAIN FORMAT = TRADITIONAL SELECT * FROM users WHERE username = 'user50000'	1 row(s) returned	0.000 sec / 0.000 sec
9	14:35:28	CREATE INDEX idx_username ON users(username)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.406 sec
10	14:36:30	SELECT * FROM users WHERE username = 'user50000' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
11	14:39:00	EXPLAIN SELECT * FROM users WHERE username = 'user50000'	1 row(s) returned	0.000 sec / 0.000 sec
12	14:40:01	EXPLAIN FORMAT = TRADITIONAL SELECT * FROM users WHERE username = 'user50000'	1 row(s) returned	0.000 sec / 0.000 sec

ANALYSIS

The experiment demonstrated that indexes significantly improve database query performance. Before creating the index, MySQL performed a full table scan, meaning every row in the users table had to be checked to find the matching username. This process was slower and inefficient for large datasets.

After creating the idx_username index, MySQL used the index to directly locate the desired row. This reduced the number of scanned rows and improved execution speed.

The experiment also showed that indexes are most effective for frequently searched columns. However, indexes consume additional storage space and slightly slow down INSERT, UPDATE, and DELETE operations because the index must also be maintained.

Queries using LIKE with a fixed prefix such as 'user5%' can still use the index efficiently, while patterns beginning with '%' cannot effectively use indexes because the database cannot determine where to begin searching in the index.

CONCLUSION

This lab activity successfully demonstrated the importance of indexes in database optimization. By comparing query execution before and after index creation, it became clear that indexes improve query performance by reducing full table scans and allowing faster data retrieval.

The activity also provided practical experience in using the EXPLAIN statement to analyze execution plans and understand how MySQL processes queries. Overall, the lab highlighted both the advantages and limitations of indexes in relational databases.